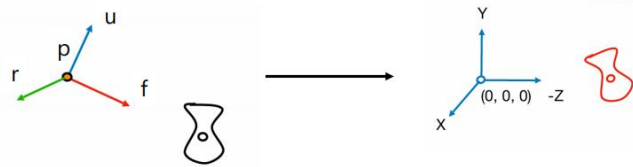


1. 填写变换矩阵，实现物体在空间中的变换，再将物体变换到相机的局部坐标和裁剪空间中

第二次作业

■ View Matrix

- 对应 look_at 函数
- 计算向量 r ，重新计算 u
- 旋转矩阵是正交矩阵



$$r = f \times u$$

$$u = r \times f$$

$$V = M_{cam}^{-1} = (T_{cam} \cdot R_{cam})^{-1} = R_{cam}^{-1} \cdot T_{cam}^{-1} = \begin{bmatrix} r_x & u_x & f_x & 0 \\ r_y & u_y & f_y & 0 \\ r_z & u_z & f_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}^{-1} \begin{bmatrix} 1 & 0 & 0 & p_x \\ 0 & 1 & 0 & p_y \\ 0 & 0 & 1 & p_z \\ 0 & 0 & 0 & 1 \end{bmatrix}^{-1}$$

$$= \begin{bmatrix} r_x & r_y & r_z & 0 \\ u_x & u_y & u_z & 0 \\ f_x & f_y & f_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & -p_x \\ 0 & 1 & 0 & -p_y \\ 0 & 0 & 1 & -p_z \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

以 look at 函数为例：

Look at 矩阵，作用是把所有世界坐标变换到观察空间——以摄像机为原点,以摄像机指向的方向为-z 方向的空间。Look At 矩阵就像它的名字表达的那样：它会创建一个看着 (Look at) 给定目标的观察矩阵。

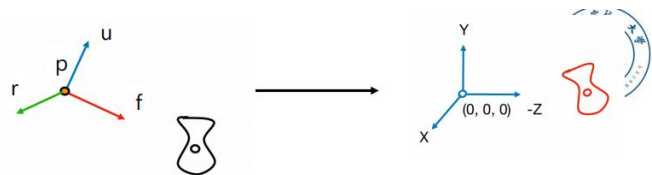
我们看到图片最下面，look at 矩阵可写成两个矩阵相乘。右侧矩阵代表摄像机位置，而左侧的矩阵则与观察空间相关， $r(right), u(up), f(front)$ 就是这个空间坐标系的三个方向。

这就是 look at 矩阵需要的，我们要得到摄像机位置，再加上摄像机自己的坐标系,也就是得到 r, u, f 。

第二次作业

■ View Matrix

- 对应 look_at 函数
- 计算向量 r ，重新计算 u
- 相机朝向 -z 方向



$$r = f \times u$$

$$u = r \times f$$

$$V = \begin{bmatrix} r_x & r_y & r_z & -p \cdot r \\ u_x & u_y & u_z & -p \cdot u \\ -f_x & -f_y & -f_z & p \cdot f \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

要得到这些我们只需在函数中传入三个参数, `pos`(相机位置), `target`(观察目标位置), `up`(世界坐标系的向上向量)

注: 传入的 `up` 可不是 `u`。

- 我们首先计算 `f(front)` 向量, 该向量从相机指向观察目标。 `f=target-pos`, 就这么简单。
- 接着通过 `f` 和 `up` 叉乘得到 `r(right)`。这里 `f` 和 `u` 不一定垂直, 但他两能确定竖直平面, 所以能确定 `r`。
- `r` 和 `f` 叉乘算出 `u`

注意叉乘讲究顺序。记得把 `r, f, u` 化成单位向量!

然后照着 PPT 填矩阵就行。

可以写个函数计算叉乘:

```
vecf3 cross(const vecf3& v1, const vecf3& v2) {  
    return vecf3(  
        v1[1] * v2[2] - v1[2] * v2[1],  
        v1[2] * v2[0] - v1[0] * v2[2],  
        v1[0] * v2[1] - v1[1] * v2[0]  
    );  
}
```

2. Phong Shading

首先咱们要清楚, 咱们接下来的工作将会在 `fragment shader` (片段着色器) 里完成, 这里是给物体上色的地方, 而我们所谓的光照、阴影, 说到底其实都是影响颜色的参数

```
vec3 lightcolor = 1.0f*point_light_radiance/255;  
color = vec3(lightcolor[0]*color[0], lightcolor[1]*color[1], lightcolor[2]*color[2]);  
color = (ambient + (1.0-shadow)*(diffuse + specular)) * color;
```

这几个语句合起来是计算 `color` 的方式。Phong shading 的计算不涉及 `shadow`。

Ambient: 环境光照参数

Diffuse: 漫反射光照参数

Specular: 镜面反射光照参数

Shadow: 阴影参数, 一般初始设为零

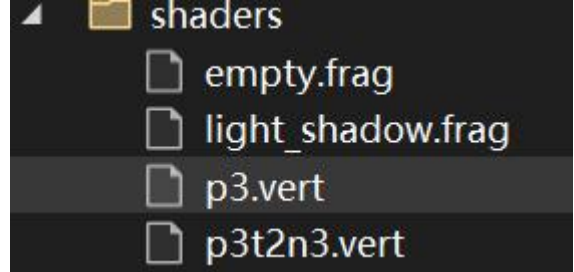
变量声明解析:

`light_shadow.frag` 孤零零晾在外面, 必然有输入输出的接口连接别的文件。out 声明的就

是输出, uniform 和 in 声明的就是输入。

in 声明的一般来自顶点着色器 (vertex shader),

看右边, 那两个 .vert 就是写顶点着色器的, 两个 .frag 的就是咱片段着色器的。



这么多 uniform, 啥意思? uniform 用于从应用程序 (通常是 main.cpp) 向着色器传递数据。uniform 是全局变量, 它可以被着色器程序的任意着色器在任意阶段访问。

看右边, main.cpp 就是这么往 frag 里传的

```
99
100     float ambient = 0.2f;
101     float specular = 0.8f;
102     auto light_pos = vec3(0.0f, 10.0f, 0.0f);
103     light_shader.set_tex("color_texture", 0);
104     light_shader.set_tex("shadow_map", 1);
105     light_shader.set_vec3("point_light_pos", light_pos);
106     light_shader.set_vec3("point_light_radiance", {200, 200, 200});
107     light_shader.set_float("ambient", ambient);
108     light_shader.set_float("specular", specular);
109
```

out vec4 FragColor; // 片段着色器, 当然输出颜色咯。除了 rgb, 还有个透明度, 所以是 vec4

uniform vec3 point_light_pos; // 光源位置

uniform vec3 point_light_radiance; // 光辐射强度, 决定了光的亮度和颜色, 分量取值在 0-255, 参与计算的时候要先除以 255

uniform sampler2D shadow_map; // 深度贴图, 后面用的

uniform bool have_shadow; // 算是一个信号吧, 辅助实现按空格显示阴影的功能

uniform mat4 light_space_matrix; // 和阴影相关的一个变换矩阵, 后面用的

uniform float ambient; // 环境光照参数

uniform float specular; // 镜面反射光照参数

uniform sampler2D color_texture; // 一个二维纹理 (可能是那个奶牛纹理), 后面会从中采集颜色值。

uniform vec3 camera_pos; // 相机位置

in VS_OUT { // 从顶点着色器传入的几个变量

vec3 WorldPos; // 目前选中的点的世界坐标

vec2 TexCoord; // 用这玩意在前面那个纹理上采集颜色

vec3 Normal; // 法向量

} vs_out; // 这给我干哪来了? 这不结构体吗? 调用方式都一样, 以 vs_out.WorldPos 这种形式调用

3. shadow mapping

```
_shadow.frag  README.md  transform.cpp  main.cpp  checkerboard.png
ndering.exe - x64-Debug  { } Utils::Transform
1      #include "transform.h"
2
3      namespace Utils::Transform {
4      |
5      matf4 perspective(float fov_y, float aspect, float z_near, float z_far) noexcept {
6      |     assert(fov_y > 0 && aspect > 0 && z_near >= 0 && z_far > z_near);
7      |
8      |     // TODO 2.1.2 : Create the perspective projection matrix.
```

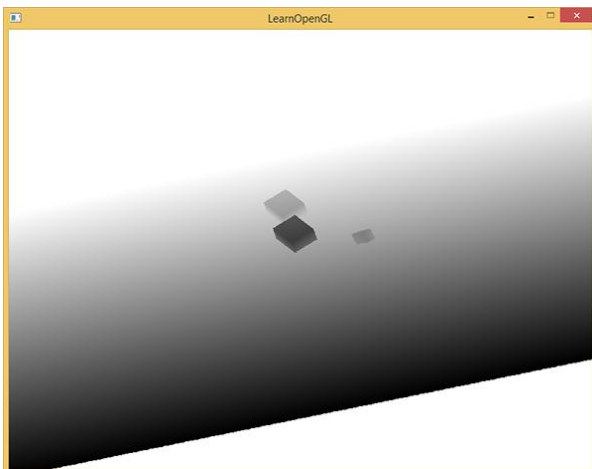
看到第三行了吗，这边创建了一个命名空间，而在命名空间后面加“::”就可以访问这个空间。比如 `std::cout << "Hello, World!" << std::endl;`

`std::`就访问了 c++库，可以用里面的函数了

所以咱们在 `main.cpp` 内调用咱们写好的函数就这么写：

```
156
157      // TODO 2.2.2 : Uncomment the following segment, then modify the light_projection and light_view
158      float near_plane = 0.1f, far_plane = 100.0f;
159      auto light_projection = Utils::Transform::orthographic(30.0f, 30.0f, near_plane, far_plane);
160      auto light_view = Utils::Transform::look_at(light_pos, plane_pos, vecf3(0.0f, 1.0f, 0.0f));
161      auto light_space_matrix = light_projection * light_view;
162      //////////////////////////////////////
```

咱们刚用这俩矩阵变换过摄像机，现在变换光源还是能用上。要不说变换矩阵好用呢。咱们变换到光源坐标系是要干一件事——生成深度贴图。简单来说，从光源的“视角”看过去，记录视野中每一个点的深度，生成一张记录这些深度的图。

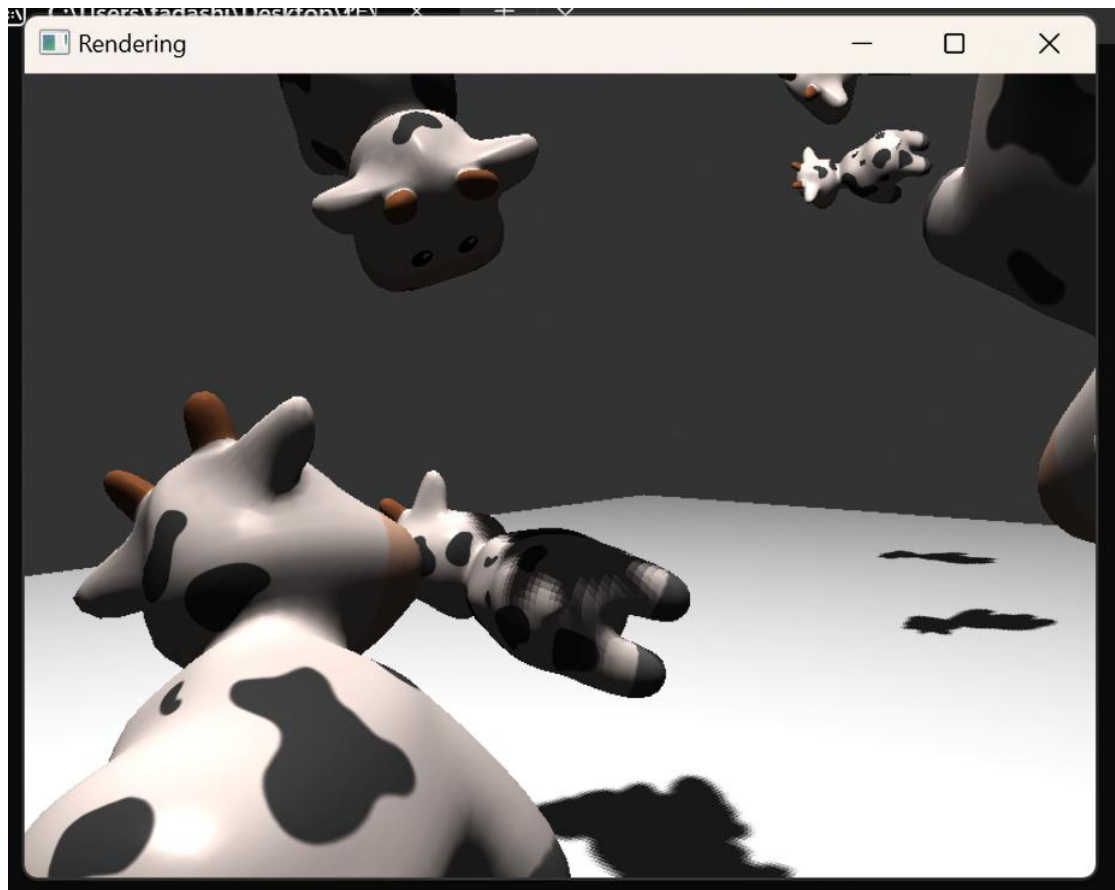


（这个在 `main.cpp` 中已经写好了）接着要做的是在片段着色器中进行判定：在对应位置，如果某个点的深度比图的深度值大，那就打上阴影。注意上面这些都是在光源坐标系下讨论的！！片段着色器要有把世界坐标变换为光源坐标系下的坐标的能力。

所以咱们把写好的变换矩阵，用 `uniform` 传到片段着色器去。就是上面的 `uniform mat4 light_space_matrix` 这句。

shadow 参数的计算要点：比较 `closest depth`（从深度贴图中取到）和 `current depth`（光源坐标系下的 `z` 值）的大小，如果后者大，`shadow=1`，否则 `shadow=0` 比较时，`current depth` 减去一个小的偏移量 `bias`，有助于消除摩尔纹。

效果展示



尝试改变 `light_radiance` 的值，光源的颜色随之改变~

